

Fleetman — Project Errors & Fixes

A running log of real errors hit while building the Fleetman project, what caused them, and how they were resolved.

Error 1 — kubernetes_manifest fails because the CRD does not exist at plan time

What is the error

While creating a **ClusterSecretStore** (an External Secrets Operator custom resource) with Terraform's `kubernetes_manifest` resource, Terraform failed on plan/apply with:

```
Error: no matches for kind "ClusterSecretStore" in group "external-secrets.io"
```

and again on destroy with:

```
Error: API did not recognize GroupVersionKind from manifest (CRD may not be installed)
no matches for kind "ClusterSecretStore" in group "external-secrets.io"
```

What caused it

- **ClusterSecretStore** is a Custom Resource (CR) whose CRD is installed by the External Secrets Operator (ESO) Helm chart.
- The HashiCorp `kubernetes_manifest` resource performs a live Kubernetes API lookup at **PLAN time** to discover the resource's schema (its `GroupVersionKind`).
- At plan time nothing has been applied yet, so the ESO CRDs are not installed → the GVK lookup finds "no matches for kind" → the plan aborts before any resource is created.
- `depends_on` does **not** help: it only controls **apply** ordering, not **plan-time** validation.

Consequences (on apply and on destroy)

- **On apply:** a fresh single apply can never succeed. The CRD is installed by the Helm release in the same run, but the manifest is validated before anything is applied — a chicken-and-egg deadlock.
- **On destroy:** even worse. The resource exists in state/config but its CRD is gone, so Terraform cannot even refresh/plan it ("failed to determine resource GVK"). This blocks the entire destroy of the layer, not just that one resource.

How we fixed it

Switched the custom resource from `kubernetes_manifest` (hashicorp/kubernetes) to `kubectrl_manifest` (the alekc/kubectrl provider):

- `kubectrl_manifest` applies **raw YAML server-side** and does **not** require the CRD at plan time.
- Created a reusable module:
Infrastructure/modules/k8s-modules/kubectrl-manifest.
- Added the alekc/kubectrl provider (pinned to 2.4.1) to the add-ons layer, configured with the same kubeconfig/context as the kubernetes/helm providers.

- Kept `depends_on = [module.helm_charts]` so the ESO Helm release (which installs the CRDs) is created first.

How it works now

- A **single terraform apply works**: Helm installs ESO + its CRDs first (because of `depends_on`), then `kubectl_manifest` applies the `ClusterSecretStore`. No plan-time CRD requirement, so no chicken-and-egg deadlock.
- **Destroy also works cleanly**, because `kubectl_manifest` never performs a plan-time GVK lookup — Terraform can plan its destruction even if the CRD is already gone.

Key takeaway: for Custom Resources whose CRD is installed in the same Terraform run (e.g. via a Helm chart), use `kubectl_manifest` (raw YAML, no plan-time schema lookup) instead of `kubernetes_manifest`.

Error 2 — terraform destroy fails: Helm release "external-secrets" not found

What is the error

During `terraform destroy`, the Helm release uninstall failed with:

```
Error: Error uninstalling release
Unable to uninstall Helm release external-secrets: uninstallation completed with
1 error(s): uninstall: Failed to purge the release: release: not found
```

What caused it

- Terraform's state still tracked the `helm_release external-secrets`, but the actual Helm release **no longer existed in the cluster** (already removed, never fully installed, or the cluster's Helm storage no longer held it).
- On destroy, the Helm provider runs `helm uninstall external-secrets`, which fails because there is no such release to purge → `release: not found`.
- In short: **Terraform state was out of sync with the real cluster** (a phantom resource).

Consequences

- The destroy **errors out on the `helm_release` resource**, leaving the layer's teardown incomplete (other resources may not get cleaned up in that run).

How we fixed it

Since the real release was already gone, we removed the stale entry from Terraform state so it stops trying to uninstall something that does not exist:

```
terraform state rm 'module.helm_charts["external-secrets"].helm_release.helm'
```

Then re-ran `terraform destroy`, which completed cleanly.

How it works now / prevention

- State now matches reality — there is no phantom Helm release for Terraform to uninstall.
- Avoid deleting Helm releases (or `kubectl delete`-ing their resources) **outside** Terraform; that desyncs state from the cluster. If it happens, reconcile with `terraform state rm <address>` for the orphaned resource, or `terraform apply -refresh-only`.